

포인터 접근 방식 실험 설계

-외부 파일의 경우-

1. Test 중점 요소

- 큰 외부 데이터를 LLM context에 직접 넣지 않아도 되는가
- pointer만 넘기고 tool이 필요한 부분만 추출하게 했을 때 성능이 유지되거나 좋아지는가
- tool output이 커질 때 runtime pointer로 넘기는 방식이 효과적인가

2. Test를 위해 필요한 것

A. pointer를 입력받는 wrapper

예:

- source pointer 받기
- runtime pointer 받기
- 실제 객체 resolve 하기

B. 필요한 부분만 추출하는 tool

예:

- 로그에서 패턴 검색
- 문서에서 관련 section 추출
- JSON field 추출

예를 들면

- `grep_log(pointer, pattern)`
- `extract_doc_section(pointer, query)`
- `extract_json_field(pointer, path)`

3. disk 외부데이터 처리 실험

예:

- 긴 문서를 통째로 넣기 vs source pointer + extraction tool
 - 긴 로그 전체 입력 vs grep tool 사용
- 이건 최소한의 extraction tool이 있어야 한다.

4. runtime pointer 실험

예:

- 큰 검색 결과를 바로 context에 넣기 vs runtime pointer로 저장 후 후속 tool 호출
- 큰 분석 결과 JSON을 통째로 넣기 vs field 추출 tool 사용

이건 runtime object store + pointer resolver + extraction tool이 있어야 한다.

즉 runtime 쪽은 source 쪽보다 한 단계 더 준비가 필요하다.

-내부 코드의 경우-

1. 코드 구조 접근 실험

- line-span selector vs symbol selector
- symbol index 유무 비교
- self-extend on/off 비교

이건 외부데이터 tool이 없어도 된다.

즉 코드 어시스턴트 핵심 실험은 먼저 돌릴 수 있다.

4. 그래서 현실적인 개발 순서는 이렇다

내가 보기엔 이렇게 가는 게 제일 현실적이다.

1단계

코드 어시스턴트 핵심부터 완성

- line-span selector
- symbol index / file outline
- symbol selector
- 코드 읽기 / 수정 tool

이걸로 먼저 코드 관련 실험 수행

2단계

disk 외부데이터용 최소 tool 추가

- 긴 로그 grep tool
- 긴 문서 section 추출 tool

이걸로 source pointer 관련 실험 수행

3단계

runtime object store와 최소 runtime tool 추가

- 큰 tool output 저장
- runtime pointer 발급
- pointer resolve 후 후속 tool 호출

이걸로 논문형 runtime pointer 실험 수행

즉 외부데이터 실험은 코드 핵심 기능 이후의 2차 실험 축으로 두는 게 좋다.

5. 발표/보고서에서는 이렇게 말하면 된다

“runtime 및 disk 외부데이터 처리 방식은 pointer 단독으로는 검증될 수 없으며, pointer를 해석하고 필요한 부분만 추출하는 tool wrapper가 전제되어야 한다. 다만 이를 위해 전체 tool 생태계의 완성이 필요한 것은 아니며, 로그·문서·구조화 데이터에 대한 대표적 최소 tool 셋만으로도 실험이 가능하다.”

이렇게 말하면 현실성과 논리성이 둘 다 산다.

6. 한 줄 결론

맞다.runtime 및 disk 외부데이터 처리 실험을

하려면 최소한의 tool 구현은 선행되어야 한다.
하지만 모든 tool을 완성할 필요는 없고, 실험 가
설을 검증할 대표적 최소 tool 몇 개만 있으면 충
분하다.

원하면 다음 답변에서

외부데이터 실험에 필요한 최소 tool 셋 3~5개
를 우선순위대로 골라주겠다.